

SEARCH1DO: Search-Based Planning in Advanced 3D Environments

Oltion Nezha

24 Gennaio, 2025

Contents

1	Introduzione e Obiettivi	1
2	Descrizione del Progetto	2
3	Struttura del Progetto	2
4	Differenze tra Algoritmi di Ricerca	2
5	Maze e Ambiente di Simulazione	3
6	Processo di Esecuzione	4
7	Visualizzazione e Performance	4
8	Considerazioni Finali e Prospettive Future	5
9	Conclusioni	5

1 Introduzione e Obiettivi

Obiettivi del Progetto

- Sviluppare un ambiente simulato per la risoluzione di labirinti.
- Implementare diversi algoritmi di ricerca (A^* , Dijkstra, Uniform Cost).
- Analizzare e confrontare le performance degli algoritmi.

2 Descrizione del Progetto

SEARCH1DO Search-Based Planning in Advanced 3D Environments

Questo progetto è stato realizzato per il corso *Introduzione all'Intelligenza Artificiale* dell'Università degli Studi di Milano-Bicocca. L'obiettivo principale è sviluppare e testare algoritmi di pianificazione basata sulla ricerca in ambienti 3D complessi, sfruttando librerie avanzate per la simulazione 3D e la fisica (ad es. NVIDIA Cosmos, NVIDIA Omniverse, Project Chrono, PyBullet).

Integrazione con Gymnasium:

- Implementazione di algoritmi di ricerca fondamentali (ad es. A*, Uniform Cost, etc.).
- Valutazione delle performance in ambienti 3D semplici.
- Stima del costo computazionale in base alle caratteristiche dell'ambiente.

3 Struttura del Progetto

Organizzazione delle Cartelle e File Principali

```
Gymnasium/  
  config.py          % Configurazioni ambiente, controller, etc.  
  main.py            % Script principale per esecuzione simulazione  
  README.md          % Documentazione base  
  requirements.txt    % Dipendenze  
  algorithms/  
    astar.py          % Algoritmo A*  
    dijkstra.py       % Algoritmo Dijkstra  
    uniform_cost.py   % Algoritmo Uniform Cost Search  
  controllers/  
    pd_controller.py  % Controllore PD per il percorso  
  envs/  
    maze.py           % Ambiente labirinto  
  utils/  
    conversion.py     % Conversione coordinate
```

4 Differenze tra Algoritmi di Ricerca

A* vs Dijkstra vs Uniform Cost Search:

- **A***: Combina il costo accumulato con una funzione euristica che stima il costo residuo fino al goal, migliorando l'efficienza della ricerca.
- **Dijkstra**: Espande i nodi basandosi esclusivamente sul costo accumulato, senza euristica. Garantisce il percorso più breve, ma può risultare meno efficiente.
- **Uniform Cost Search**: Simile a Dijkstra, espande i nodi in ordine crescente di costo accumulato. In questo progetto, è particolarmente utile per gestire terreni differenti, poiché consente di assegnare costi specifici (ad es. acqua, sabbia, strada) in base alla difficoltà di attraversamento.

5 Maze e Ambiente di Simulazione



Figure 1: Labirinto utilizzato nella simulazione.

Il labirinto viene creato tramite una funzione che sfrutta `Gymnasium` per registrare e istanziare l'ambiente. Nel file `maze.py` vengono definiti due tipi di labirinti:

- **LARGE_MAZE**: un labirinto standard.
- **LARGE_MAZE_CUSTOM_TERRAIN**: un labirinto con terreni personalizzati, dove vengono specificati costi differenti per tipi di terreno (ad es. acqua, sabbia, strada). Quest'ultimo viene utilizzato dall'algoritmo Uniform Cost per gestire il costo variabile di attraversamento.

La funzione `get_maze(custom_terrain)` permette di scegliere quale labirinto utilizzare, mentre `get_maze_dimensions(maze)` restituisce il numero di righe e colonne. Inoltre, i parametri per il rendering (come larghezza, altezza e seed) e per la configurazione dell'ambiente sono definiti all'interno del file.

6 Processo di Esecuzione

Come Eseguire il Progetto

- Installare le dipendenze:

```
pip install -r requirements.txt
```

- Avviare la simulazione specificando l'algoritmo:

```
python Gymnasium/main.py --algorithm astar
```

Dettagli del Processo di Simulazione

- **Ricerca del percorso:** Dopo aver calcolato i percorsi in modalità discreta, il percorso viene ottimizzato per velocizzare la simulazione.
- **Conversione in coordinate continue:** Il percorso ottimizzato viene trasformato per guidare l'agente nell'ambiente.
- **Controllo proporzionale-derivativo:** Il PD Controller gestisce la navigazione lungo i waypoint estratti dal percorso.

7 Visualizzazione e Performance

Metriche e Visualizzazione

- Viene riportato il tempo di esecuzione e il numero di nodi espansi per ogni algoritmo.
- La simulazione mostra il percorso realizzato nel labirinto attraverso la funzione di rendering di Gymnasium.

8 Considerazioni Finali e Prospettive Future

- Implementazione di ulteriori algoritmi di ricerca per confronti più estesi.
- Gestione avanzata degli errori e introduzione di ulteriori metriche di performance.
- Possibile integrazione con altri ambienti di simulazione 3D (PyBullet, Nvidia Omniverse e Cosmos).

9 Conclusioni

Il progetto SEARCH1DO rappresenta una solida base per la sperimentazione e lo sviluppo di algoritmi di pianificazione in ambienti 3D complessi. L'integrazione con librerie avanzate e l'uso di terreni personalizzati, in particolare per gestire costi variabili nell'algoritmo Uniform Cost, offrono spunti interessanti per ulteriori ricerche e miglioramenti futuri.